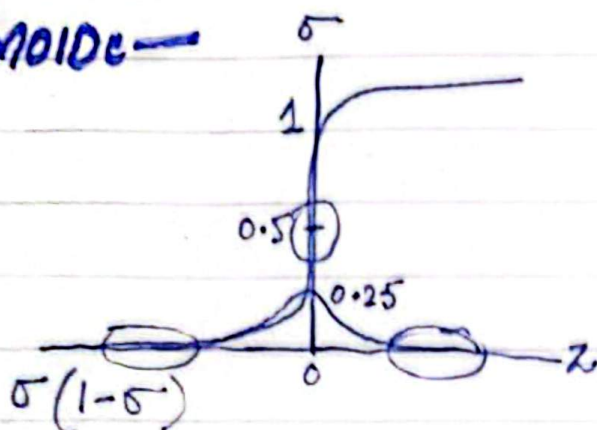


Signs of vanishing G.D:

→ Heights are not changing as we go backward.

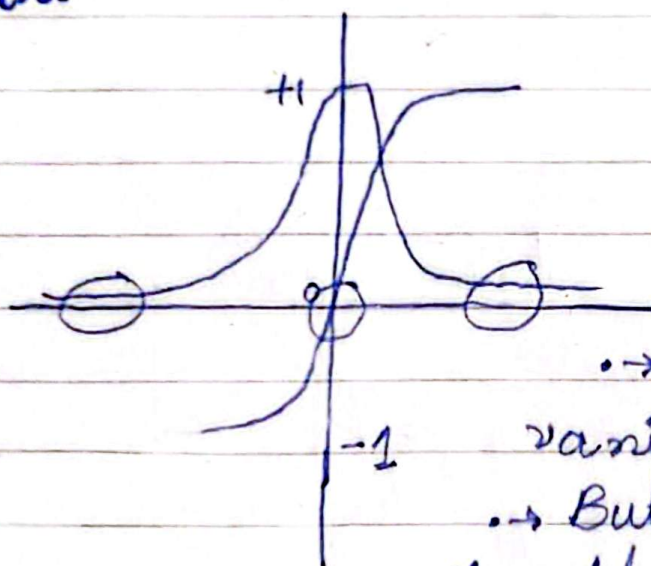
Recap: — → causes vanishing G.D.

Sigmoid —



Saturating
of

tanh:



$1 - f(z)^2$

→ also causes

vanishing G.D.

→ But better than sigmoid bcz its max value at derivative is 1.

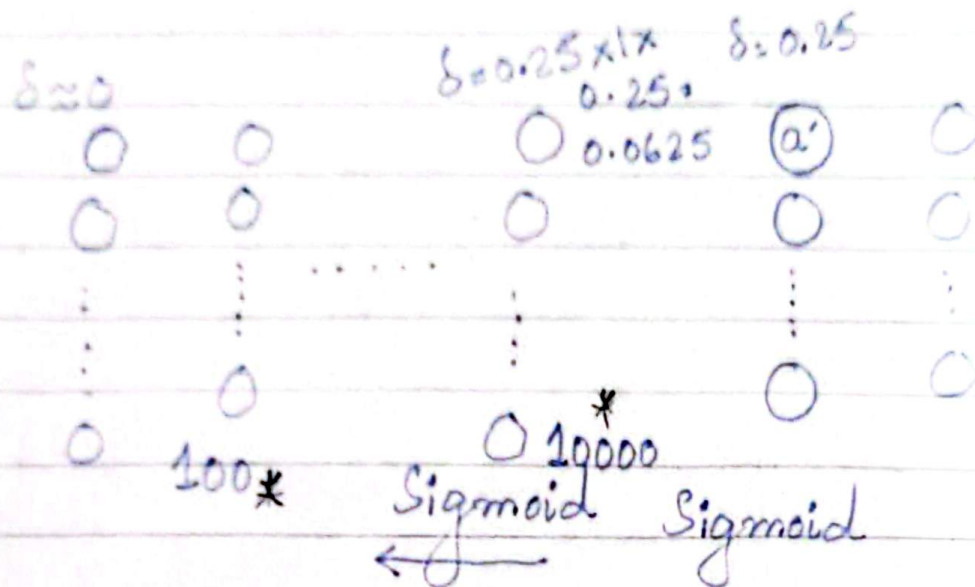
ISSUES IN ANNs —

→ In sigmoid, if a' is 0.25 (max) and weight is 1 and the network is deep, in backpropagation $\delta \approx 0$, which will cause gradient to be 0 (model is prematurely trained).

→ If weights are taken high, the gradient will start to expand or can explode i.e. cross limit of floating point range.

Yousha Bengio
Xavier Glorot
* about V.G.D.

Estimating of
weight initialization



* Weight initialization $\approx$ No. of nodes

$f_{in} \rightarrow f_{in}^* 100$ # of i/p's to a node
perceptron node

$f_{out} \rightarrow$ # of nodes of current layer
 $* 10000$

.) He proposes some weight initialization techniques.

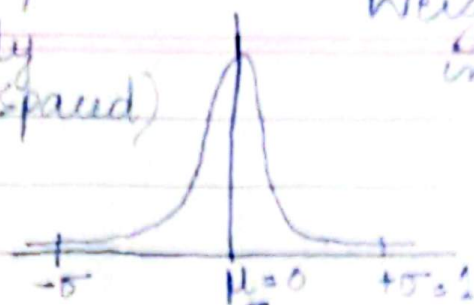
WEIGHT INITIALIZATION:—

→ We can... some weights for each node, beg in that it will produce some pattern of o/p's.

(not evenly spaced)
Gaussian/Normal:

$$G(\mu, \sigma)$$

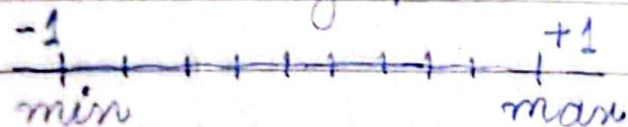
$$N(\mu, \sigma)$$



$$SN(0, 1)$$

↳ Standard Normal distribution.

Uniform: (evenly spaced)



→ it will generate evenly spaced values.

Global Initialization:—

$$weight = \frac{1}{\sqrt{f_{in}}} \left[-\frac{1}{\sqrt{f_{in}}}, \frac{1}{\sqrt{f_{in}}} \right]$$

$$Weight = \mu \left[-\frac{\sqrt{f_{out}}}{\sqrt{f_{in} + f_{out}}}, \frac{\sqrt{f_{out}}}{\sqrt{f_{in} + f_{out}}} \right]$$

↳ default (Normalized)

↳ ReLU is used in deeper networks.

mostly
for RNN

He initialization:

→ Gaussian.
weight = $\mathcal{G}\left[0.0, \sqrt{\frac{2}{n_{in}}}\right]$

→ Used in exploding G.O, we clip G.O after Gradient Clipping: certain value.

Batch Normalization:

→ Normalize/Scale the o/p's (Tanh layers or the layers that we want).

We can normalize,

→ Inputs of some i/p layers.

→ Hidden layers.

1) post-normalization.

→ tanh, sigmoid.

2) pre-normalization i.e on z (logit).

→ (before applying af).

then
→ apply af.

→ apply normalization after af.

Sequential [

for pre normalization → Input (dim),
Dense (128),
Batch Normalization(),
Activation ('tanh').

for post normalization → Dense (128, 'relu'),
Batch Normalization(),
Drop Out (0.5)]

SPECIALIZED NNs—

Flattening of Image Data:

Default image dataset

storage = 4D.

image 1.

i.e 1, 2
3, 4

$$\left[\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \right]$$

coloured.

→ before giving
it to NN we

convert 4D into 2D.

2D

$$4D \rightarrow \begin{bmatrix} [] & [] \\ [] & [] \\ [] & [] \end{bmatrix}$$

i/p's

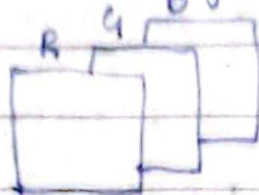
0 to NN

→ image 1 (1 row)

SHORTCOMINGS OF ANN'S —

1) Spatial Information Loss. → Due to
reason: flattening of ^{array} images.
enhancement: CNN (convolutional NN)

can take 3D image as it is.
NN loses the referencing of images as
in CNN we create 3D (for each colour, →
colour depth)



2) Poor Scaling:

reason: full connectivity / dense
enhancement: CNN (we do not
change whole connections).

3) Cannot handle seq. data:

→ Sometimes we req. connection
(relation) b/w previous o/p's & current
o/p's such as video of running person,
text data (sentence) formation.

enhancement: RNN (recurrent NN).

Image { VGG16 → Classifier
 Net 10,000 { Inception Resnet162 →
 dataset better option (162)

TRANSFER LEARNING →

→ Tweaking on pre-trained model acc. to our needs (less cost).

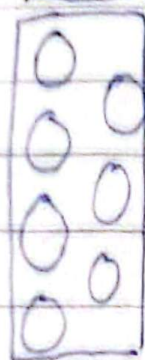
What is transferred?

1) Architecture.

2) Weights.

Why it works?

→ In images, mostly basic features are same, so we don't need to retrain starting layers.



classifier.

0 → o/p.

Don't need to retrain

→ freeze.

→ unt trainable.

source domain → pre-trained data.

(model is pre-trained on cheetah ho).

target domain → your data

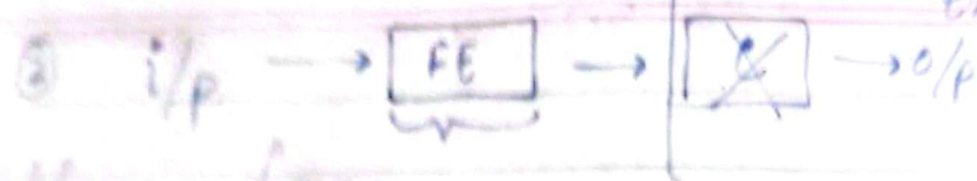
(give model to train human ho).

3-ways,

1) Use a pre-trained NN as a classifier.



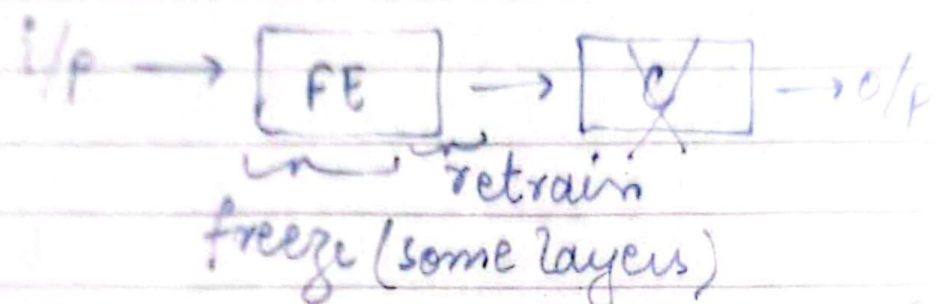
if source & target are similar domain.



If freeze

Source & target domain are somewhat similar

③ Fine tuning:

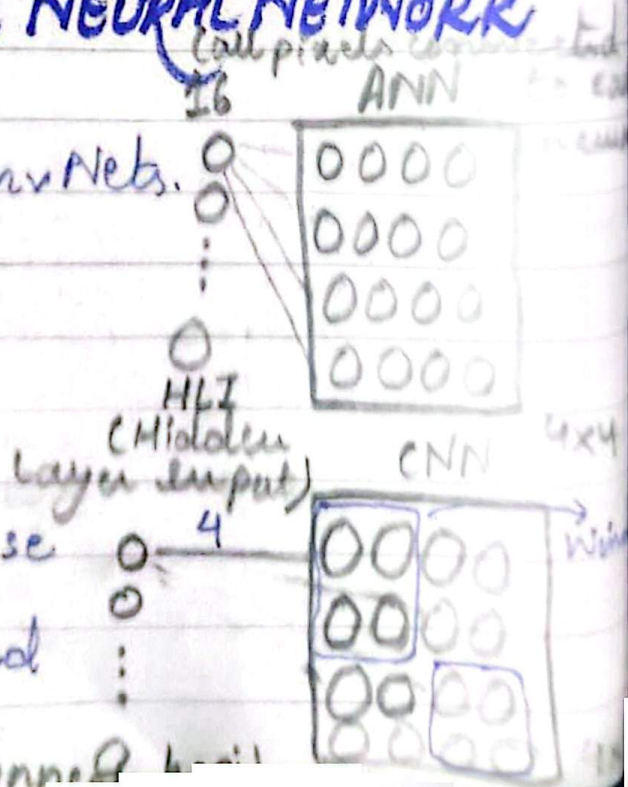


- hyperparameter (what layers to freeze & unfreeze). → need to see target domain
- we can initialize retraining from pretrained weights.

CONVOLUTIONAL NEURAL NETWORK (CNN's) :-

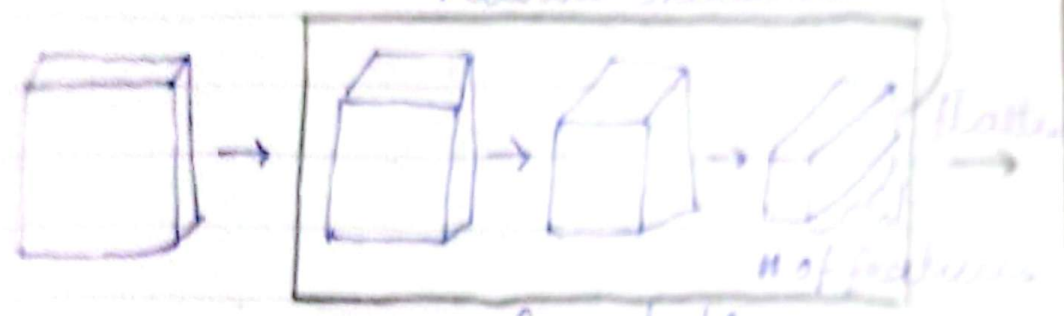
→ Also called Conv Nets.

- * → pixels behave as (image) feature size
- the windows fixed
- category. take time
- max out window connect



→ Filter ke hum image mai slide karvate
 rahinse aur jahan par wo features mil
 jayen to feature map mai likh denge

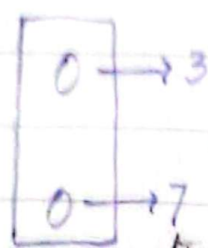
ARCHITECTURE → ^{like a deck of cards} each card is a feature
 Feature Extractor



Classifier



Output



Convolutional Layer
 (Convolution +

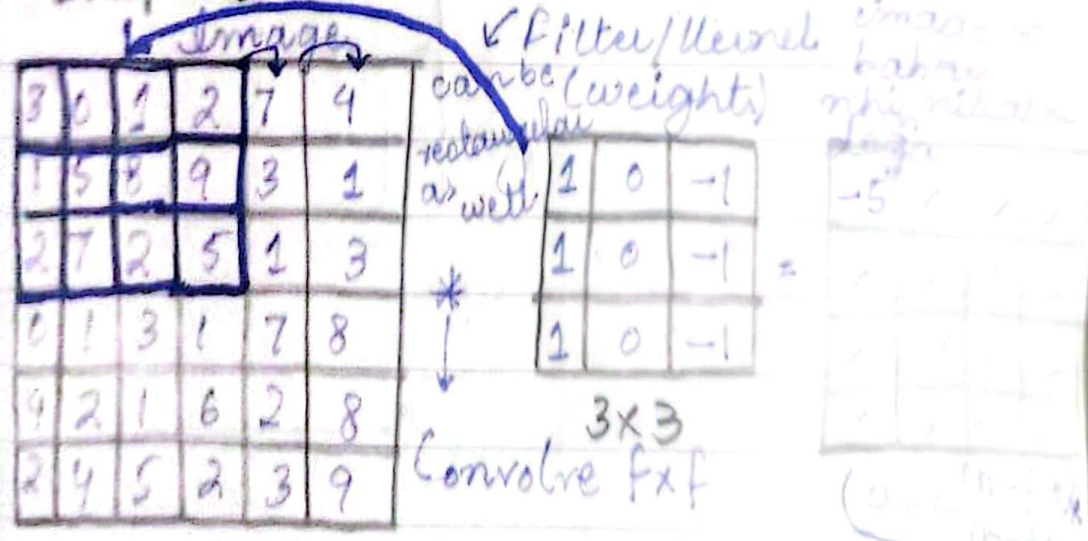
Pooling)
 w, h ↓
 & d ↑

→ aisi image ko FM dena
 FM ke liye i/p hai
 typically 1-3 layers aur 6

Fully connected layer

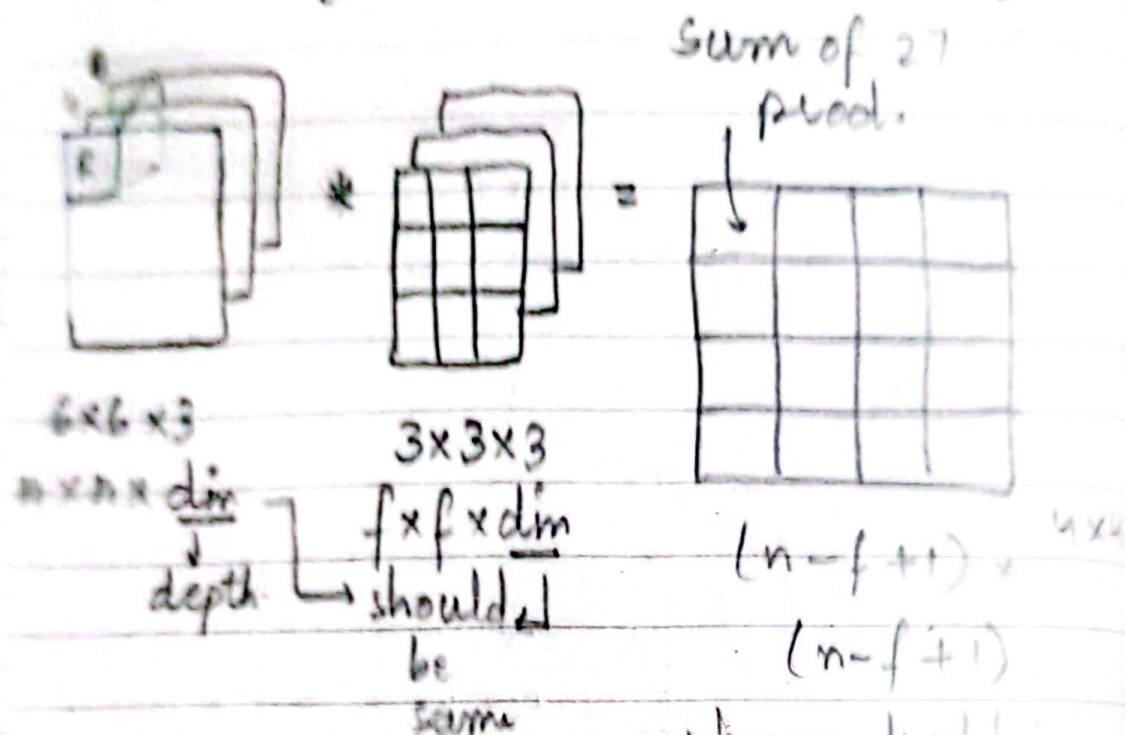
→ Feature Extractor takes image as 3D
 and through out convolutional layers
 the images remains 3D only w, h & d ↑

Convolution Operation: — ^{Filter ke aise aise}
^{Example} ^{line aise} ^{slide krke}



3x1 + 0 + (-1) + 1 + 0 + (-1) + 2 + 0 + (-2) = -5

Example: 2. (Convolution over volume)



Hyperparameters:

1) Padding:

In python: 'valid' = no padding
'same' = Feature map size same as input (FEM)

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

$6 \times 6, p=1$

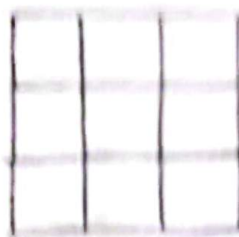
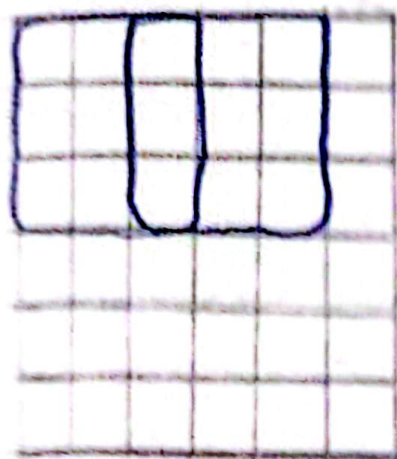
$n \times n, p$

$f \times f$
 3×3

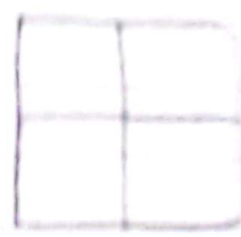
$(n+2p-f+1)$
 $(n+2p-f)$
 $= n$

padding

2. Strides
(3)



Filter
 $f \times f$



$F \times F$

$6 \times 6, p=0, s=2$

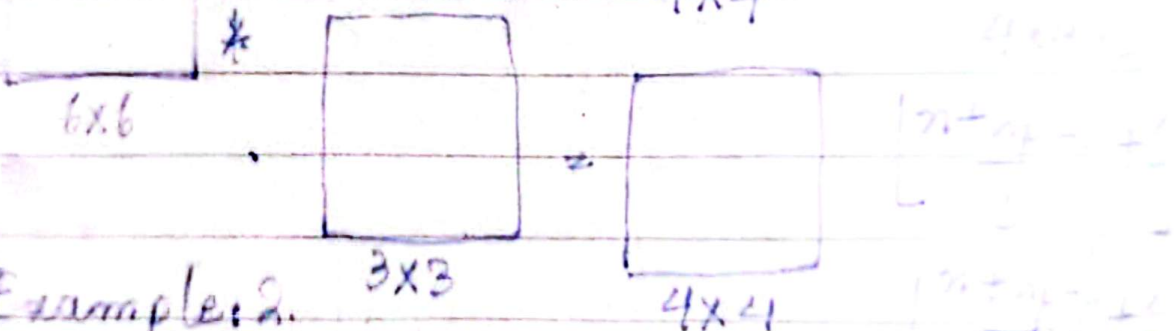
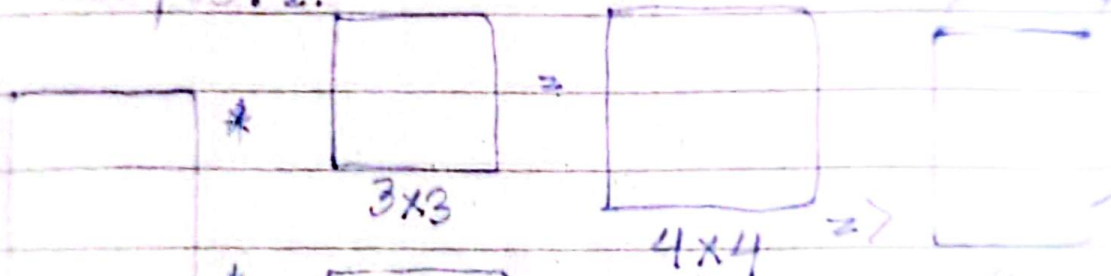
image $n \times n$

$$\left\lfloor \frac{n+p-f}{s} + 1 \right\rfloor$$

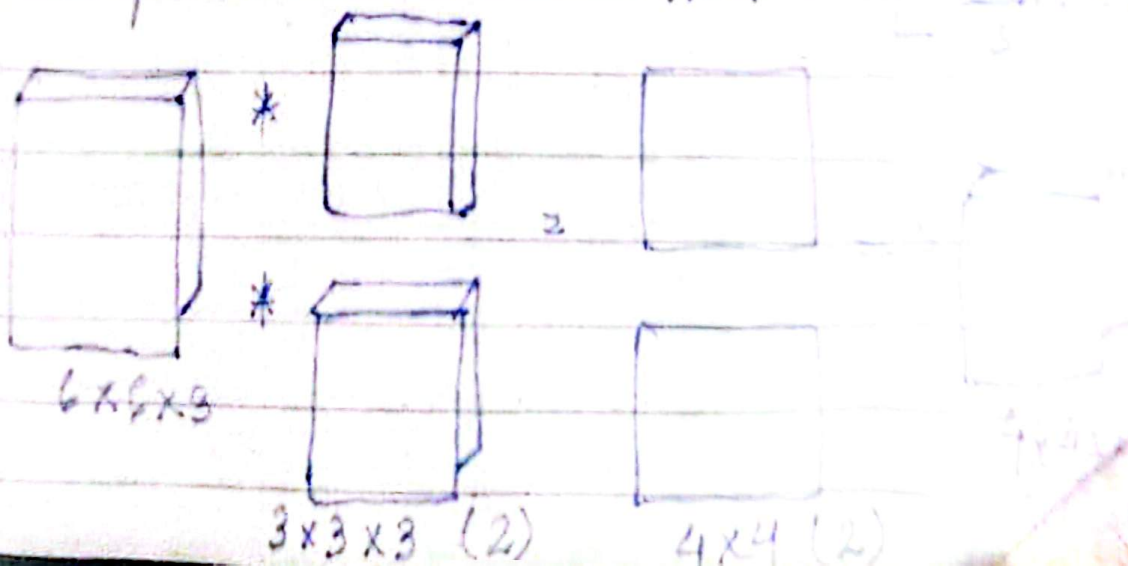
$$\times \left\lfloor \frac{n+p-f}{s} + 1 \right\rfloor$$

3. Number of filters:

Example: 1.



Example: 2.

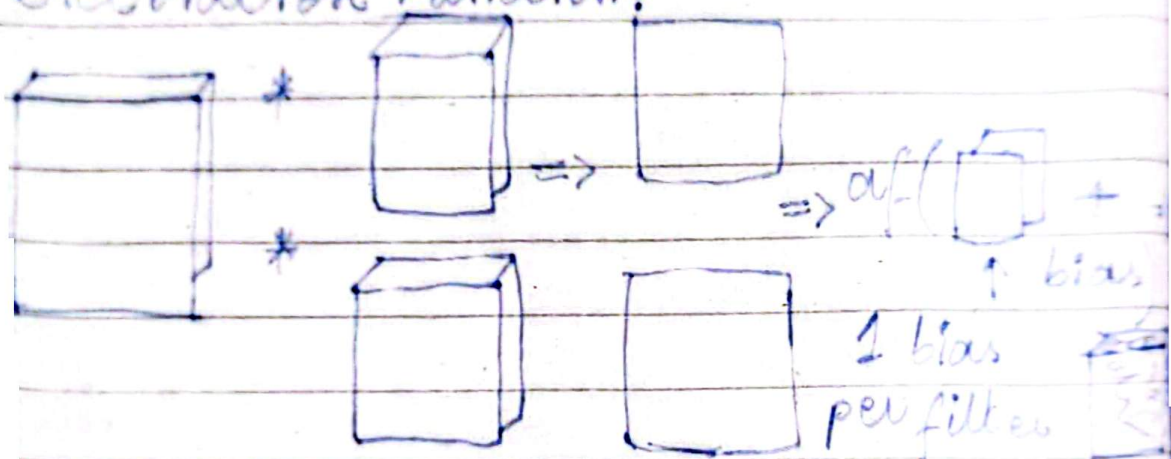


* No. of filters = Computational cost + Weights?

1) Filter Size:

Common sizes are 3×3 & 2×2 .

Activation Function:



$\rightarrow$ Weights:

$$\text{Filter Size} \times \text{No. of filters} + \text{Bias/no. of filters} = 56$$

$(3 \times 3 \times 3) \quad (2) \quad (2) \quad = 56$

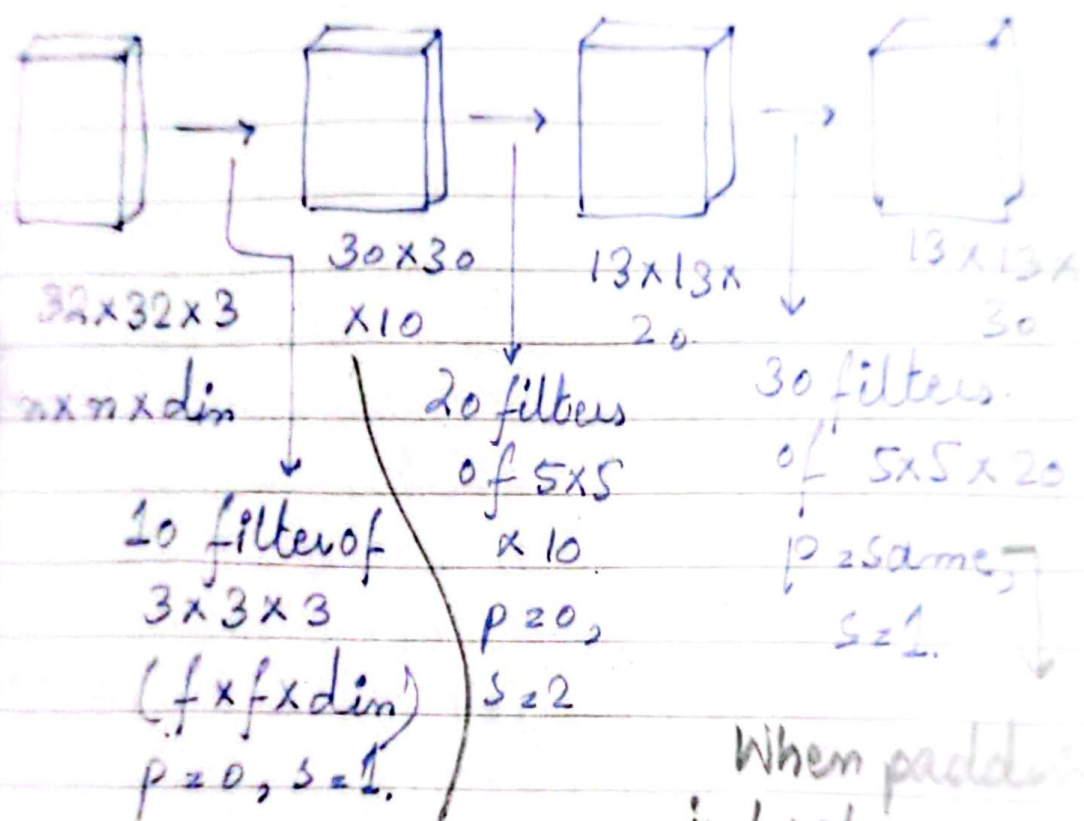
Exercise:

Find no. of weights:

Input	Filter	No. of parameters
6x6	3x3	$9+1=10$
8x8	3x3	$9+1=10$
6x6	2 of 3x3	$3 \times 3 \times 2 + 2 = 20$

$6 \times 6 \times 3$	2 of $3 \times 3 \times 3$	$3 \times 3 \times 3 \times 2 + 2 \times 56$
$6 \times 6 \times 3$	10 of $3 \times 3 \times 3$	$3 \times 3 \times 3 \times 10 + 10 \times 280$

Cascaded Convolutional Layers.



$$\left\lfloor \frac{n + 2p - f + 1}{s} \right\rfloor$$

When padding is kept same the height & width i.e size would remain same.

→ pooling to reduce height & width of
max pooling → default F.M.

→ In 2D FM hi depth create hogis
pooling will be applied on all depth's F.M.
and it will reduce dimension i.e height &
width.

→ Depth remains unchanged.

→ We usually apply convolutional operation
and then apply pooling.

→ If pool

Hyperparameters:

Filter size.

→ Stride.

→ Pooling type.

→ $f=3$ & $s=2$ se image bahar ilera
initial jayega but it will be non zero.

→ Modern architectures don't use pool
filter.

→ Unlike convolutional layer, pooling
layer's filter is ^{just} used to reduce width
& height.

→ We can also applying multiple conv
layer & then pooling layer.

VGG
weight layers.

→ Pooling is applied on the feature map ~~if~~ created by convolutional layer.

→ For classification, we use our traditional fully connected layers on flattened image but unlike ANN's we do not flatten image as soon as we get i/p.

if $28 \times 28 \times 10$ image is used as i/p in FC, 40 features are taken as i/p.

→ The i/p's given to FC is usually already extracted and we just need to apply classification.

i/p size $n \times n \times \text{dim}$ $\xrightarrow{\text{pooling filter}}$ $\left\lfloor \frac{n-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n-f}{s} + 1 \right\rfloor \times \text{dim}$

- Input size is not uniform or different
- ^{intra}Independencies

RECURRENT NEURAL NETWORKS (RNNs) —

- Time-based data. → independencies
- forecasting (involves time stamps) (relation b/w training examples)
- Non time based sequence data.
- ↳ independency (within an example) i.e. audio, text.
- Examples:

Application	Input	Output
1) Sentiment Analysis	There is nothing to like in this book.	1-5 (star reviews) -ve / +ve / neutral (comments)

Machine Translation	How are you?	؟ كيف أنت
---------------------	--------------	-----------

Name Tagging	I met Sara & Ali.	I met Sara & Ali. (Identify names).
	human : prob. high name	non " " : low prob.

Language Modeling	I would like to have	I would like to have water.
	contextual info. is	

→ have RNN's mac word by word i/p
 before unlike in CNN's & MLP's for image
 tabular data bcz there is an order b/w

Text Encoding:

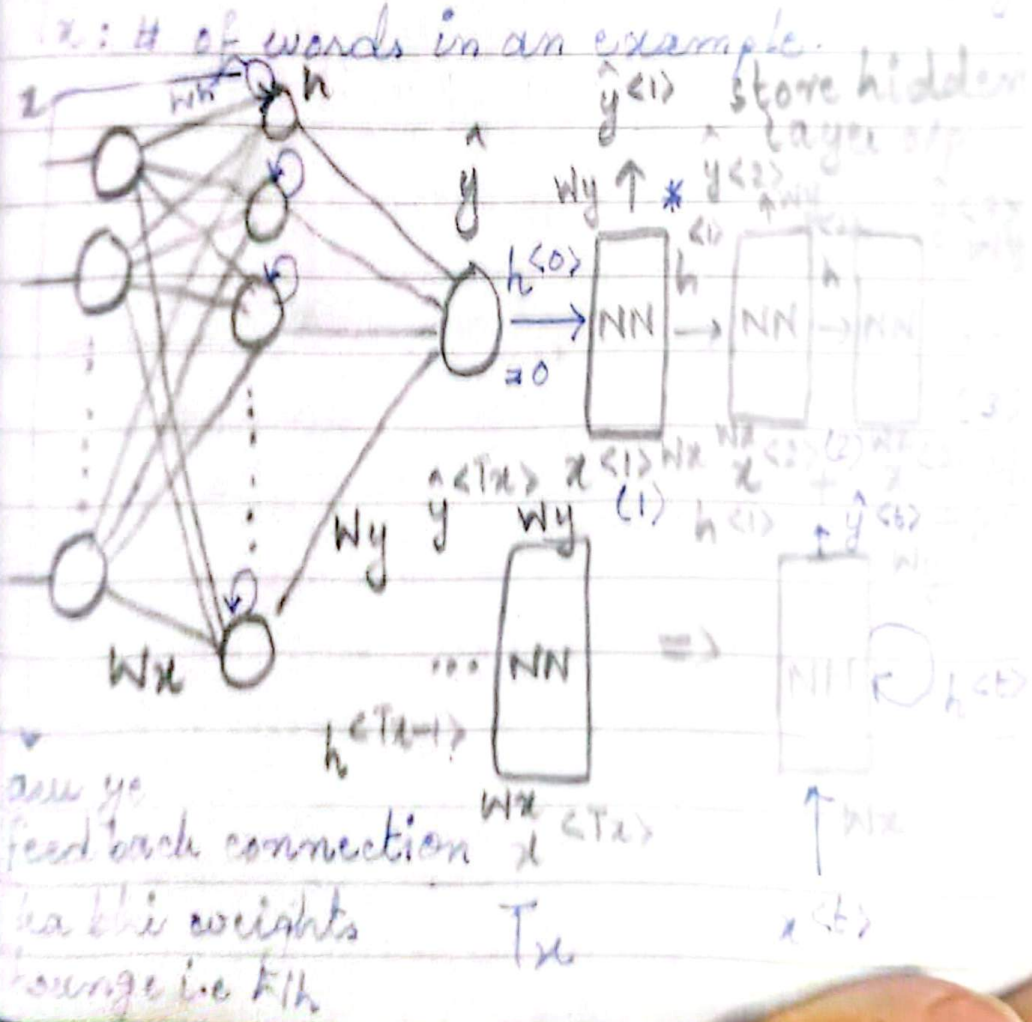
Example: Cat eats cheese

→ After tokenization and all

• $[encode('Cat'), encode('eats'), encode('Cheese')]$ → how word is along
 word embeddings • $\begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}, \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}, \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}$

Text Notation:

sample # → layer no.
 $x_i^{(l)}[t] \leftarrow t$ → word #
 j → feature no. * save o/p of hidden layer



In eqs:

$$h^{(t)} = af(x^{(t)}, h^{(t-1)}) = af(x^{(t)} W + h^{(t-1)} W + bias)$$
$$\hat{y} = af(h^{(t)}) = af(h^{(t)} W_y)$$

MEMORY:-

(8-10 words in example)
Short term: RNN's have short term memory (forgets earlier words).

Cats, which is brown in colour, eat cheese.

Long term:

Enhancements:

- LSTM (long short term memory).
- GRU gated (recurrent unit).

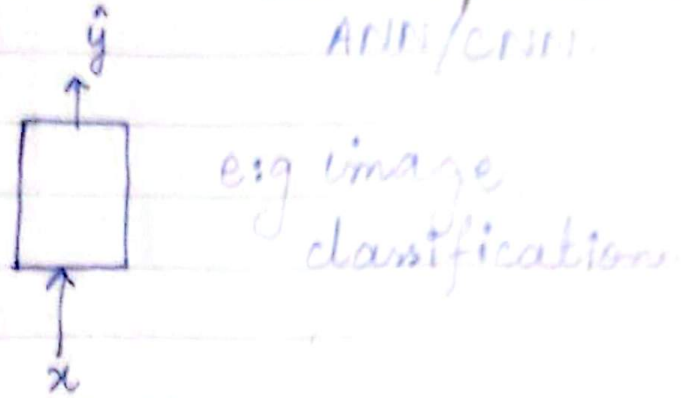
Transformers.

Image captioning $\rightarrow$ Describing content of image using textual representation

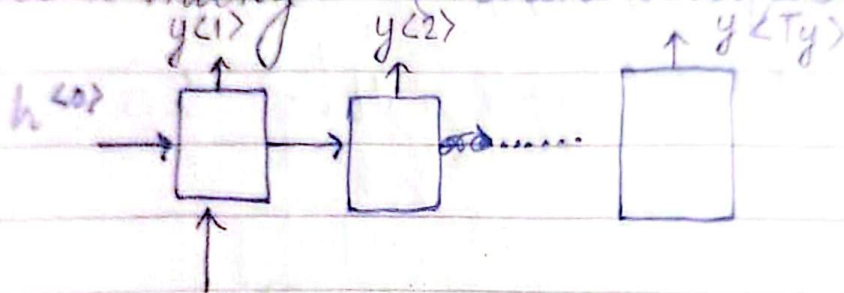
TYPES OF RNNs ON THE BASIS OF I/P O/P SEQUENCE —

One to one: $\hat{y}$ x $y(1)$ $y(2)$ $y(T_y)$ $x(1)$ $x(2)$ $x(T_x)$ $\hat{y}$

ANN/CNN



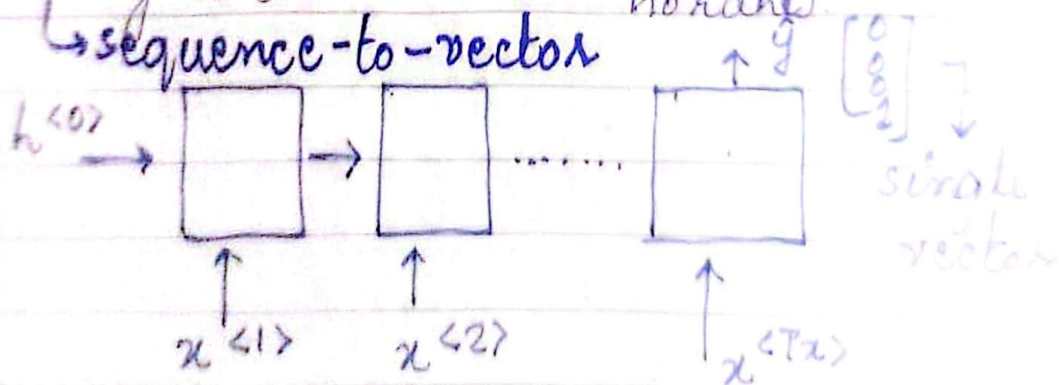
One to many: — (vector to sequence)



x (poora image aile soth dedia tor tor lie nhi lie in text)

e.g. Image captioning $\rightarrow$ poora single vector generate horake

Many to one: $\rightarrow$ sequence-to-vector



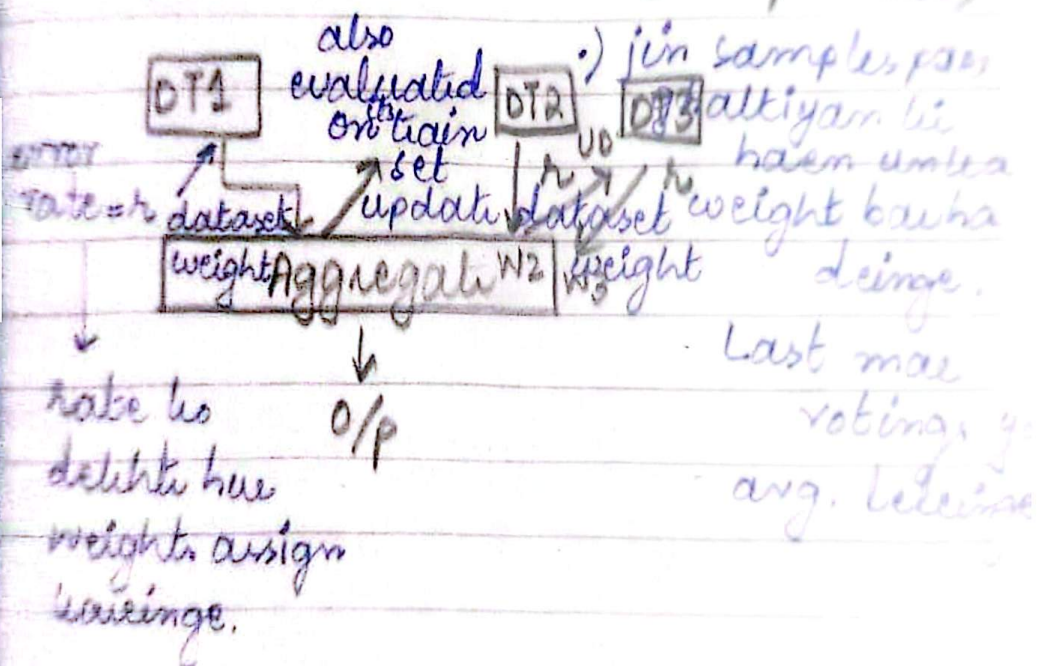
e.g. sentiment analysis.

samples are leftover samples after sampling which can be used for validation.

- BOOSTING:** → learns from previous.
- Typically built with DT learners.
 - Learners are trained sequentially.
 - Ada boost.
 - Gradient boosting.

Adaboost:

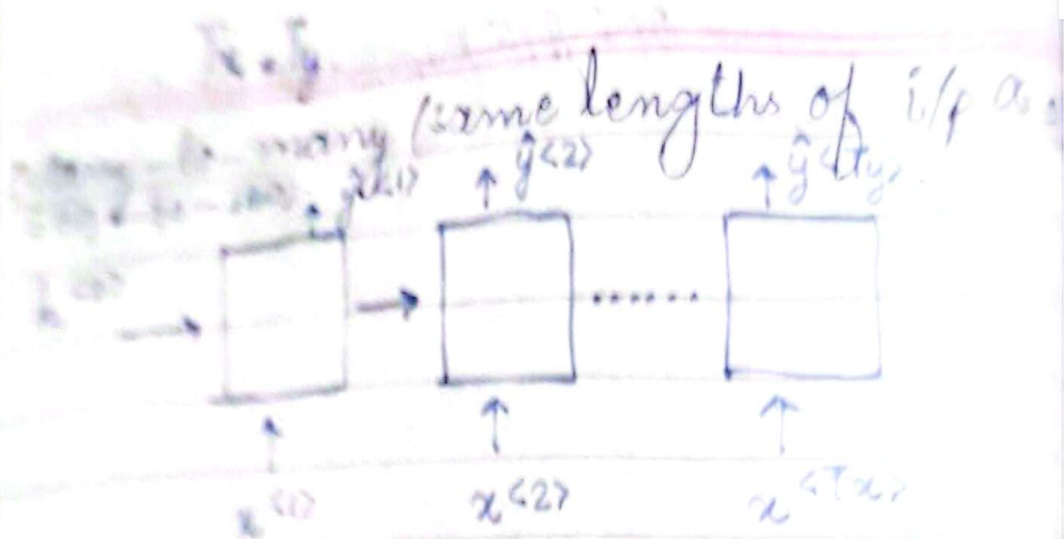
- Adaptive boosting.
- (weights of samples & algo are updated).



(error rate α) weight

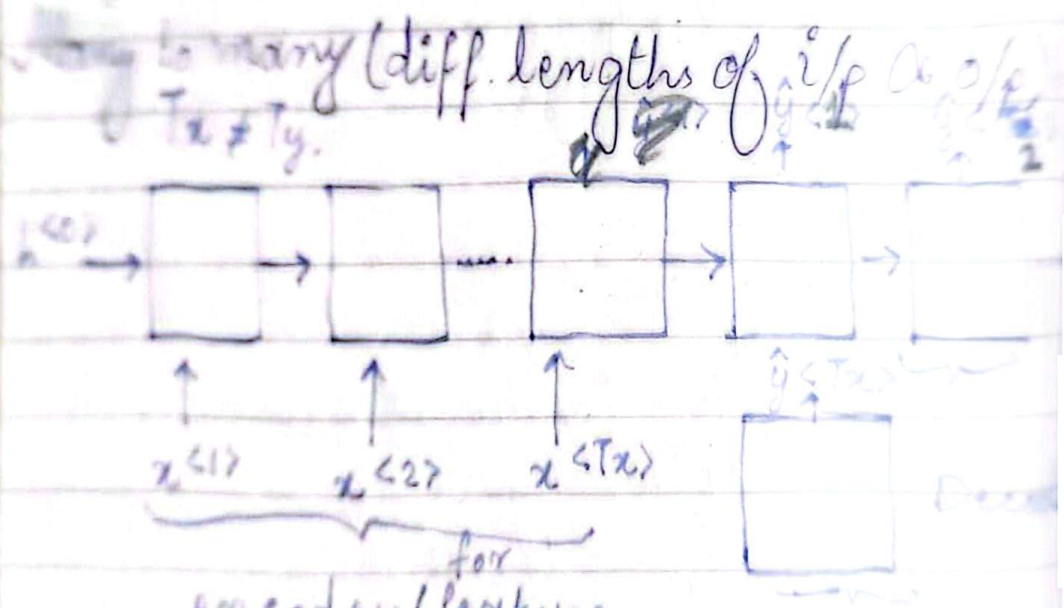
GRADIENT BOOSTING:

- We improve the residual error and we'll try to predict these errors i.e. y_2 would be the r_1 (residual error).



Seq name tagging.

Encoder Decoder.

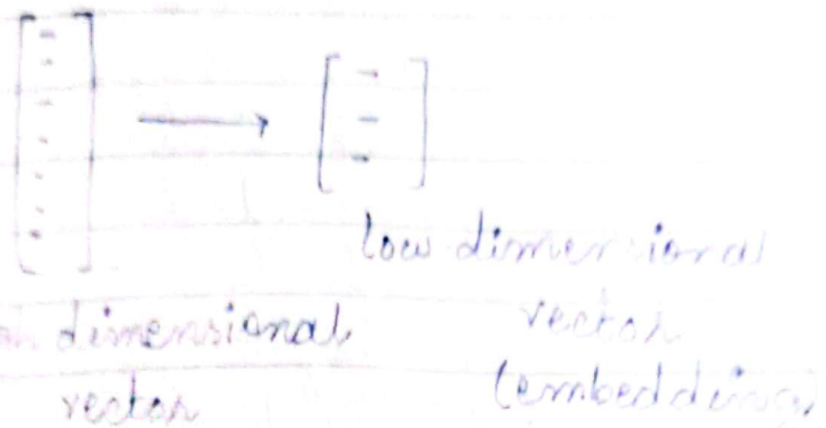


esg. encoder (feature extraction) → input seq. pphl
 → Machine translation. encode both then
 decode translation

→ human words are like vectors
 distance b/w through distance (jo features
 similar hain unka distance kam hoga)

EMBEDDINGS IN DEEP LEARNING:-

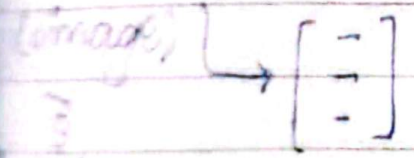
→ embeddings maps high dimensional
 vector to a low dimensional vector



Two types:

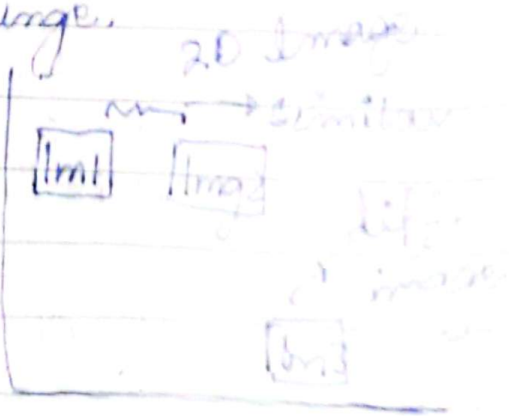
→ word embeddings: singular/plural
 has word → has vector
 → gender / men

→ visual embeddings:



word embeddings

wo feature words ke jo similar
 honge wo aapke apne
 men → queen place honge.
 embedding gender ②
 space (latent space)



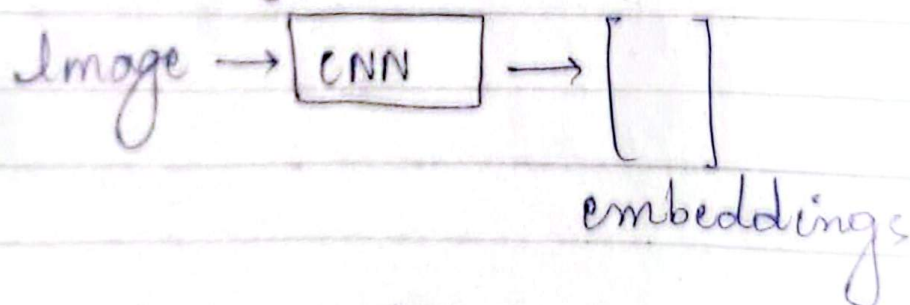
One shot learning:

→ network ko face ki embedding banana silkhana (for image identification).
uske liye jabki human face lesakhe, we don't need 1000 images of a single person for training, only one image is req. of each person, then will be able to identify a person because network knows how to create embeddings, aur agar wo person hese class us jagah se nhi hota toh it means we do have embeddings of that person.

e.g.

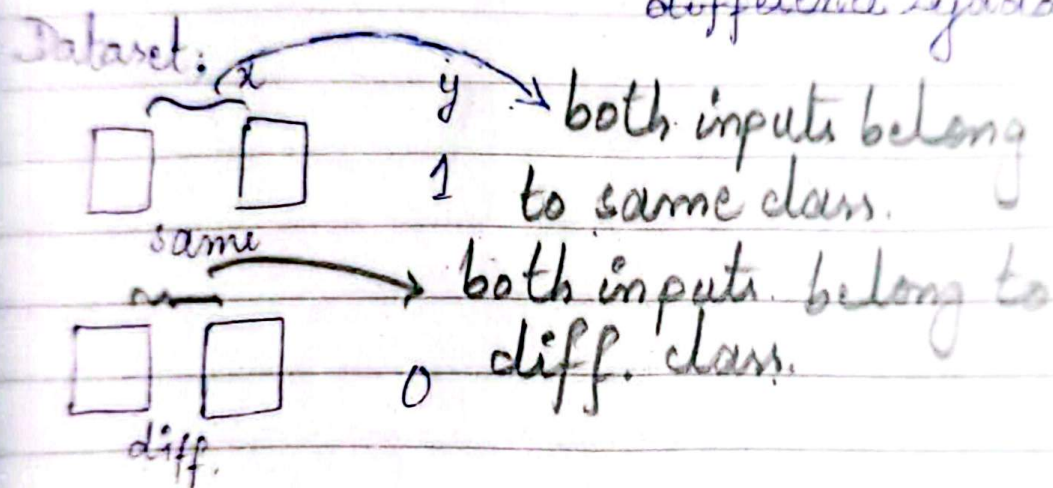
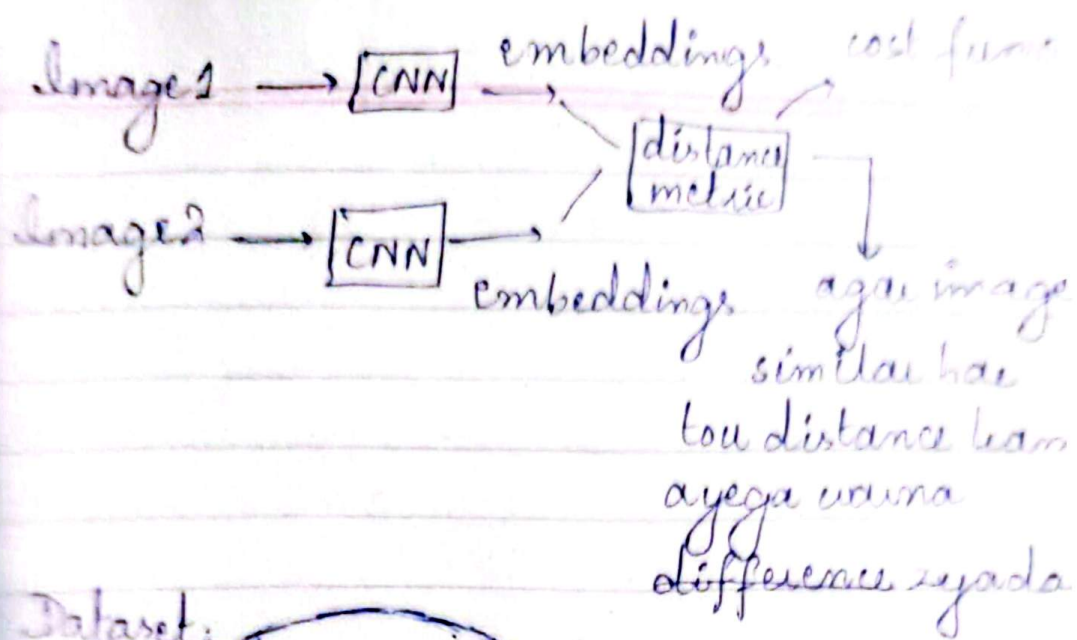
→ Face recognition, image matching, object re-identification, finger print/signature matching.

for training: CNN.



→ We train CNN's to create embeddings.

* Siamese N/W:



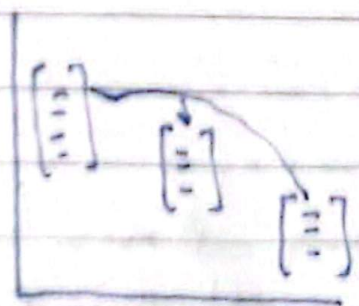
→ Once trained, we don't need twin n/w's we can use single n/w because it knows now how to create embeddings.

Main goal: Latent

achi

go.

→ Given human image like vector like for example
average out latent in to we can create a new
image.



Latent space

→ Difference of two vectors i.e. concept
vector.

→ Kabhi kabai aesa hota hai ki ham ko images me interpolation karin aur kisi garbage image aaye. [garbage]

AUTOENCODERS: — (Unsupervised Learning)

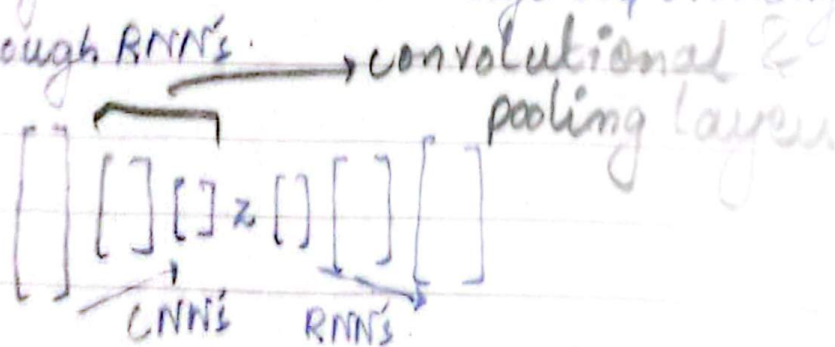
Input → Encoder → Embeddings (Compressed) → Decoder → Reconstructed Input

→ We need to make images similar even after compressing.

Hum usually 2 ho embeddings use karte hain aur then we will decode it into the image similar to original image. (Difference (MSE) should be small, otherwise weights should be adjusted.)

Decoder uses Upsampling or transposed convolutional.

→ Hum pehle embedding generate karte hain through CNN's & then image captioning through RNN's.



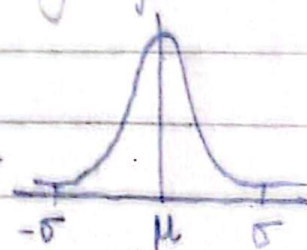
Issue in Autoencoders: Unsmooth latent space representation (empty spaces in latent space) due to which smooth transition with diff. embeddings is not possible & it results in invalid or unrealistic output.

VARIATIONAL AUTO ENCODER $\rightarrow$

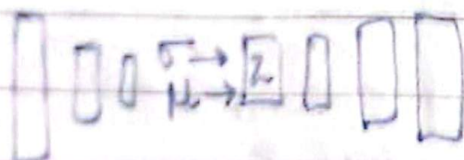
- $\rightarrow$ Generative & probabilistic model.
- $\rightarrow$ Instead of creating a vector 'z' we create a distribution i.e. range of values.

Normal/Gaussian.

- $\rightarrow$ a vector of σ & a vector of μ .



- $\rightarrow$ Has feature h_i corresponding dist. h_{z_i}



So, $z = \mu + \epsilon \cdot \sigma$ $\rightarrow$ a tensor of small values.

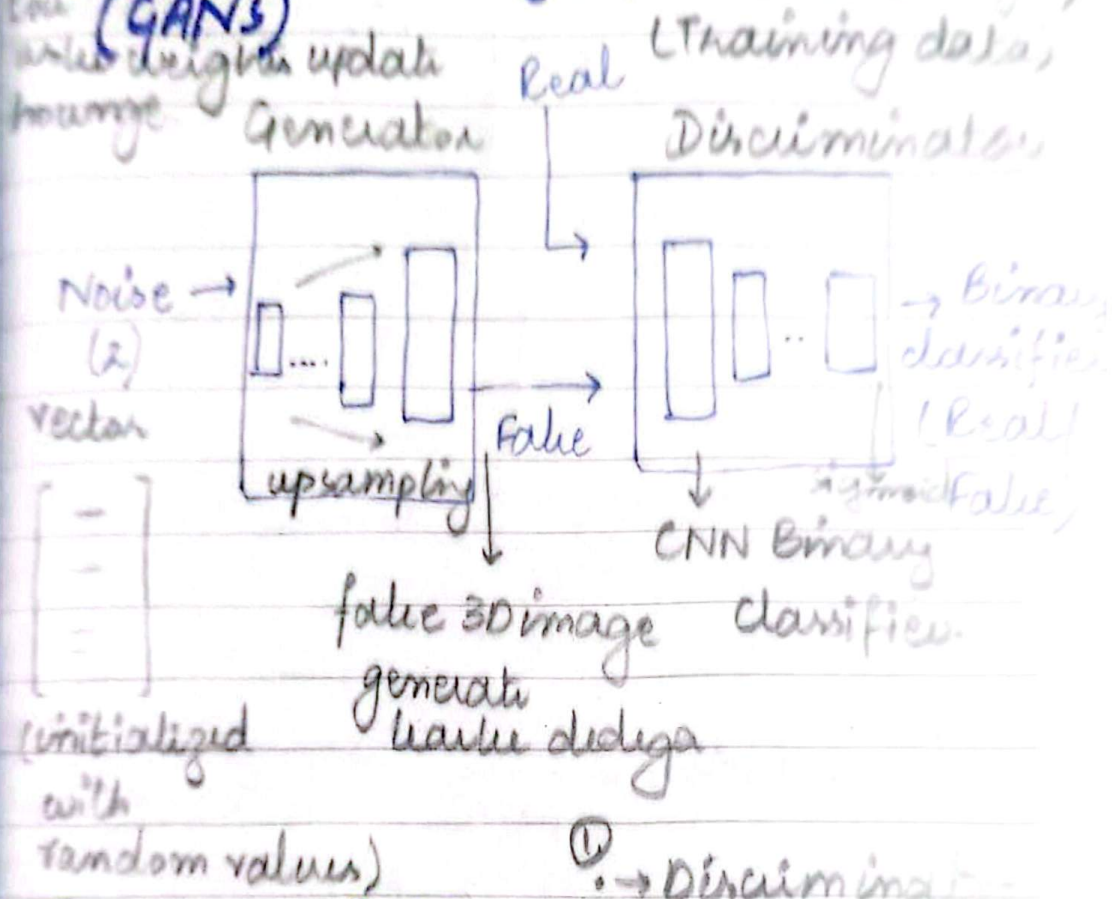
$\rightarrow$ aile loss func. z par lagega jo enforce karega μ & σ aile known gaussian curve kase kase is applied to z off z (reconstruction error)

Agar discriminator ne fake pehchan lia to generator ke weights update honge (punish) agar discriminator fake classify nhi krta to generator ke weights update honge (reward)

for example: 2 players

Competitive

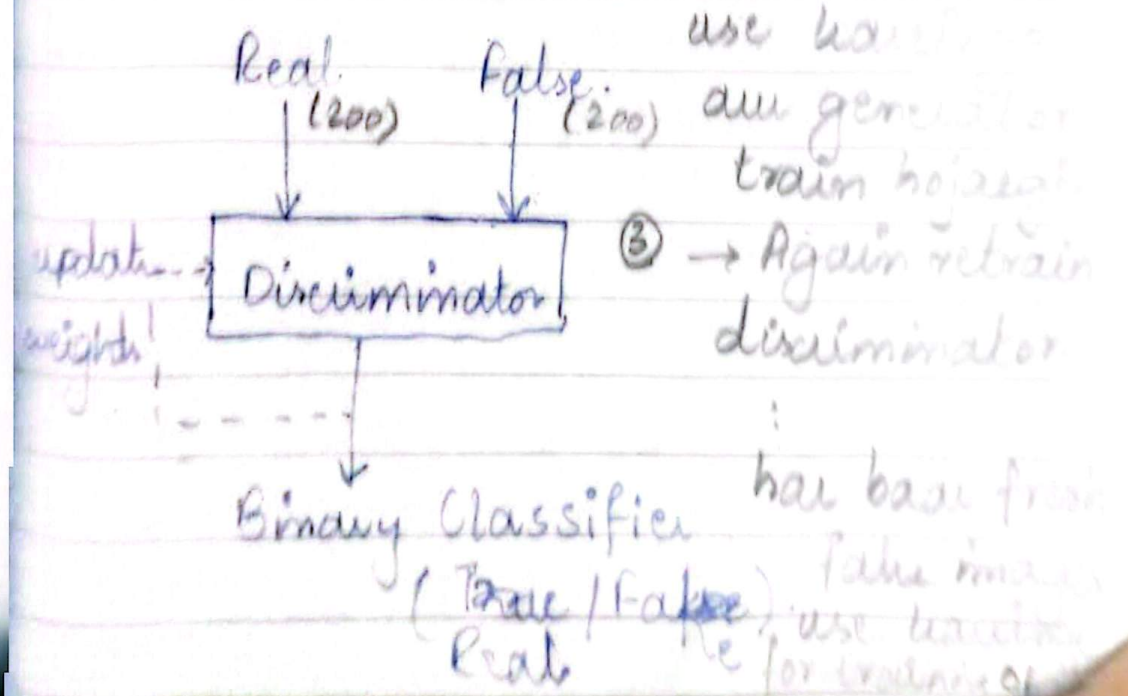
GENERATIVE ADVERSARIAL NETWORKS (GANs)



Training a GAN:-

1) Discriminator:-

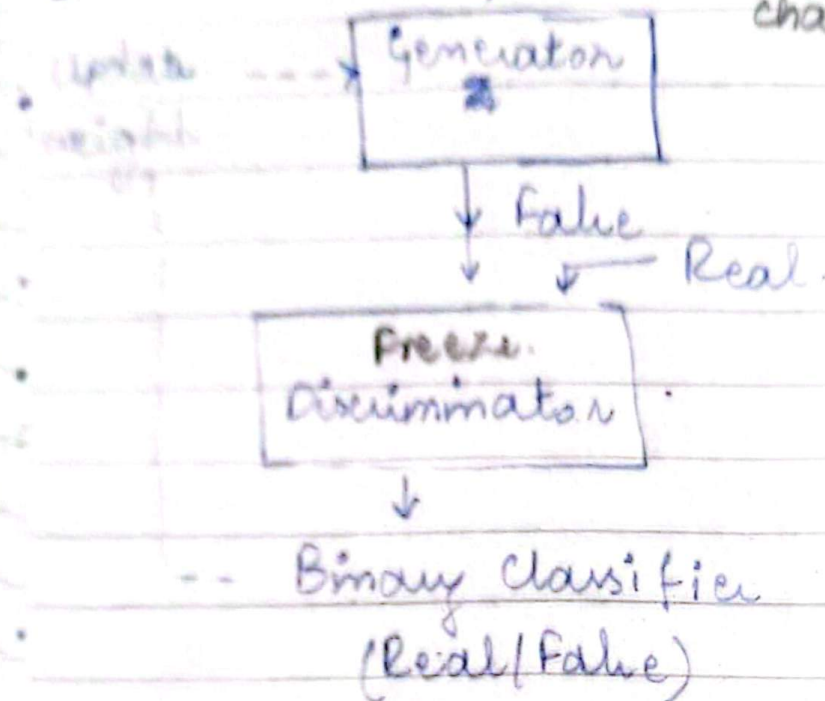
- ① → Discriminator train krke real images pr.
- ② → Generator ko use krke fake images generate krke aur generator ko train krna.
- ③ → Again retrain discriminator



→ Generator generating false images.

y for

→ Generator at training like live
Generator: z → **discriminator** like change.



Loss Function:

(min max func.)

$$\min_G \max_D V(G, D) = E[\log(D(x))] + E[\log(1 - D(G(z)))]$$

↑ D (both the terms) ↑ 1st ↑ 2nd term
 generator discriminator 1st 2nd term

GAN Evaluation

- 1) Manual Inspection only.
 ↳ Inspection by dentist
 Field inputs i.e. dental data
- 2) Inception method by dentist
 ↳ Inception

Application

- 1) StyleGAN (super resolution GANs)

for improving reso. of image

→ Image is given to user for checking that it looking like actual

(training starts from low resolution image)

•) cGAN (control GANs).

Pix2Pix.

•) Style GANs $\rightarrow$ text-to-images.

Lazy Learning / Memory-based Learning

K-NEAREST NEIGHBOR (KNN)

KNN Classifier

Stores all training examples without any computations.

1) When a new sample 'X' arrives.

• Identifies 'k' nearest neighbors.

• Identifies the class C_i which is most frequent among the k nearest neighbors.

• Assigns C_i to x.

A ← B — Manhattan
A ← B — Euclidean

Distance Metrics:

1) Euclidean Distance:

$$d(A, B) = \sqrt{\sum_{i=1}^m (a_i - b_i)^2} \rightarrow (L2 \text{ norm})$$

where,

$A = (a_1, a_2, \dots, a_m)$ & $B = (b_1, b_2, \dots, b_m)$

2) Manhattan Distance:

$$d(A, B) = \sum_{i=1}^m |a_i - b_i| \rightarrow (L1 \text{ norm})$$

for making Manhattan distance

3) Minkowski Distance:

$$d(A, B) = \left(\sum_{i=1}^m |a_i - b_i|^p \right)^{1/p} \rightarrow (Lp \text{ norm})$$

general form for distance

$p=1 \rightarrow$ Manhattan

$p=2 \rightarrow$ Euclidean

Example:

Training example	feature vector	y (Class)	Dist.	y (Target)
1	[1, 3, 1]	1	$\sqrt{(1-2)^2 + (3-4)^2}$ $\uparrow \sqrt{(1-2)^2} = \sqrt{1}$	1
2	[3, 5, 2]	1	$\sqrt{(3-2)^2 + (5-4)^2}$ $+ (2-2)^2 = \sqrt{2}$	6
3	[3, 2, 2]	0	$\sqrt{(3-2)^2 + (2-4)^2}$ $+ (2-2)^2 = \sqrt{5}$	8
4	[5, 2, 3]	0	$\sqrt{(5-2)^2 + (2-4)^2}$ $+ (3-2)^2 = \sqrt{14}$	10
x	[2, 4, 2]	?		

Weights

$$1/\sqrt{2}$$

$k=1$ $x=1$ Since $\sqrt{2}$ is smallest.

$$1/\sqrt{3}$$

$x=1$ Since $\sqrt{2}$ & $\sqrt{3}$

$$1/\sqrt{5}$$

smallest & both are 1.

KNN Regressor. -

1) Finds k neighbours.

2) Avg. the target values of these k neighbours.

3) Assigns the avg. to x.

$\Rightarrow$ If $k=1$, $y = 0$ of x.

$\Rightarrow$ If $k=2$, $y = (4+6)/2 = 5$ of x.

Weighted Nearest Neighbours:

1) assigns weights wrt distance to new samples:

$$w_i = 1/d(x, x_i)$$

$\because$ the smaller the distance, the more important

classification $\rightarrow$ higher combined weight of each
regression $\rightarrow$ find weighted avg.

Hyperparameters:

k

binary class $\rightarrow k$ odd (to avoid tie).

multiclass $\rightarrow k \geq 7$ * We will need to
 $C_1 \geq 3$ use some tie breaker
 $C_2 \geq 3$
 $C_3 \geq 1$

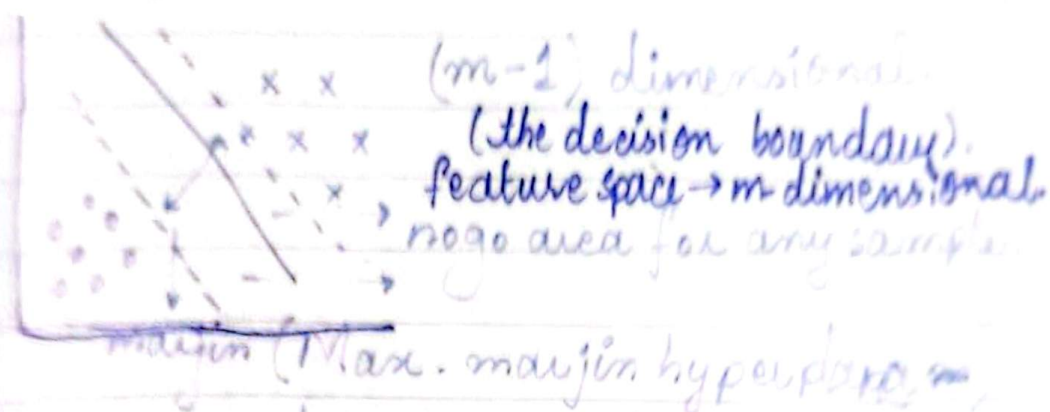
Data Preprocessing: -

Always use scaled data (normalized & regularized)
 $\rightarrow$ irrelevant feat. shouldn't be there (should be removed).

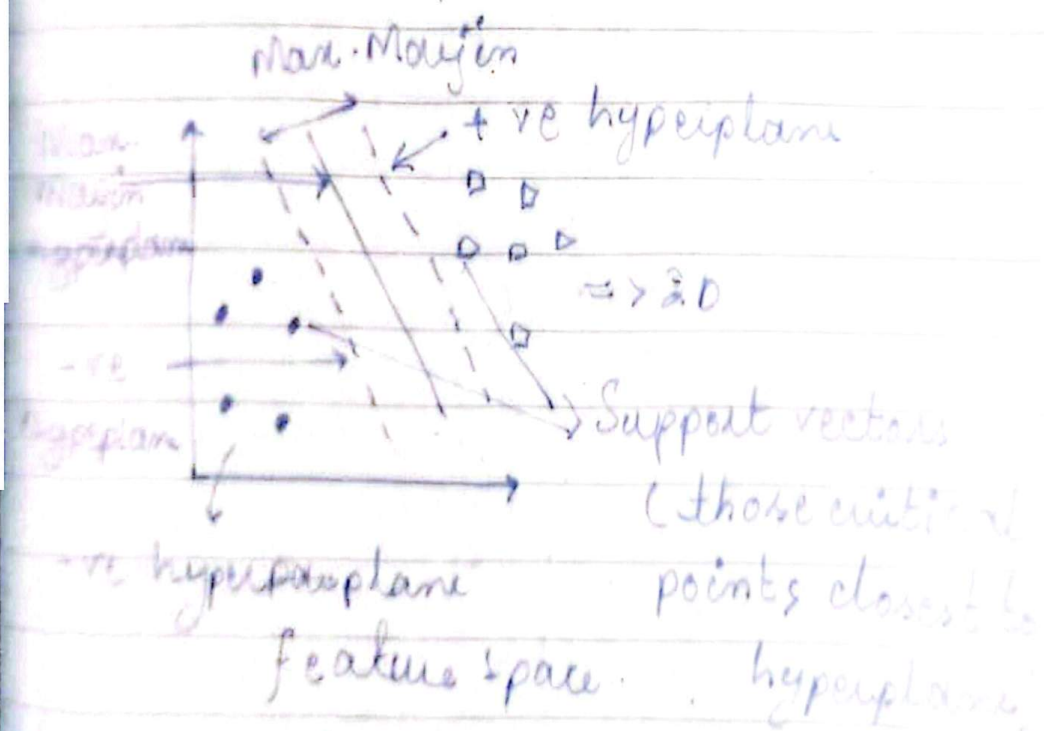
- Decision boundary can be changed by changing hyperparameter of an algo. also
- good for small dataset

SUPPORT VECTOR MACHINE — (large margin classifier)

- SVM finds that decision boundary which has max margin (distance b/w two hyperplanes always perpendicular to decision boundary)



- In a nonlinear SVM, classes are not separable using straight line.

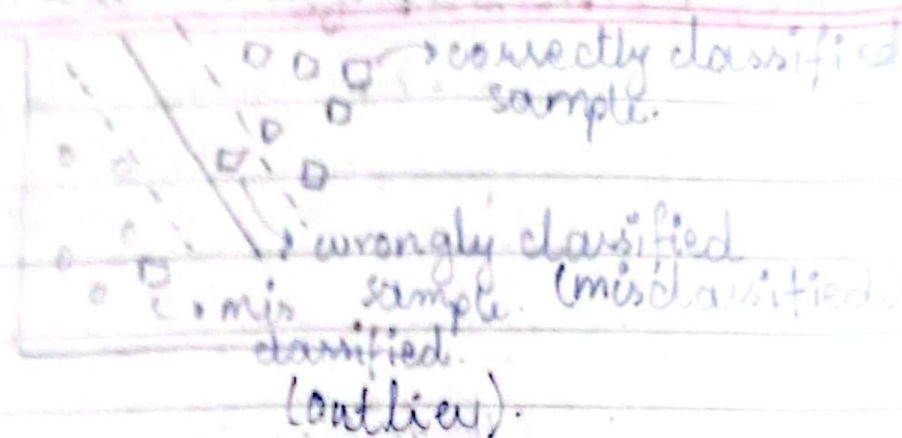


- Decision boundary will remain same unless support vector is changed.

Margin

(class 0 or 1)

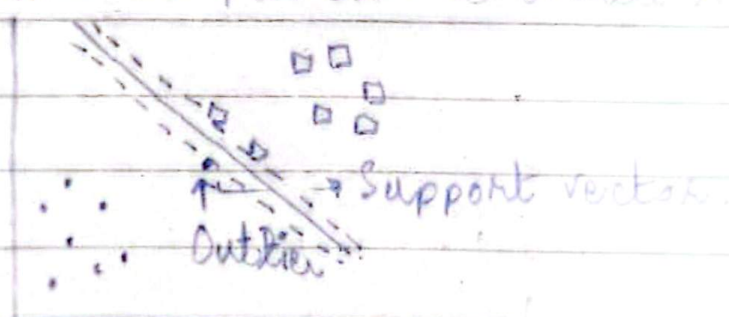
Hard margin $\rightarrow$ All the sample hyperplane is and in a row.



But in reality, margins are small,

decision boundary are not properly visible. so hard margin is not possible in this case.

Soft margin $\rightarrow$ Some margin violation is possible (some samples can be inside margin).



Hypothesis func. : — $w^T x + b \geq 0$ (class 1) or -1
 $w^T x + b = 0 \rightarrow$ +ve values (+ve class)
 $-ve$ values (-ve class)
 weight vector w input x bias b
 (class 0) or -1

$$-1 \text{ or } +1 \quad w^T x + b < 0$$

$$h(x) = \hat{y} = \text{sign}(w^T x + b).$$

$\rightarrow$ No prob. only hard o/p.

$$D.B. \text{ eq. : } w^T x + b = 0.$$

$\| \cdot \|_2$

Strict Hard margin $\rightarrow$ Overfitting

The margin boundaries are given by

$\rightarrow$ it doesn't mean lie margin is
 $w^T x + b = 1$ & $w^T x + b = -1$ Euclidean distance 1

$\downarrow$ $\downarrow$ boundary b
+ve support li. -ve support
hyperplane eq. right hyperplane eq.
side

If we allow some outliers in margin $\rightarrow$ we
can get wider margin (In soft)
margin.

Obj. Functions:

- $\rightarrow$ Max. margin.
- $\rightarrow$ Penalizes incorrect classification.

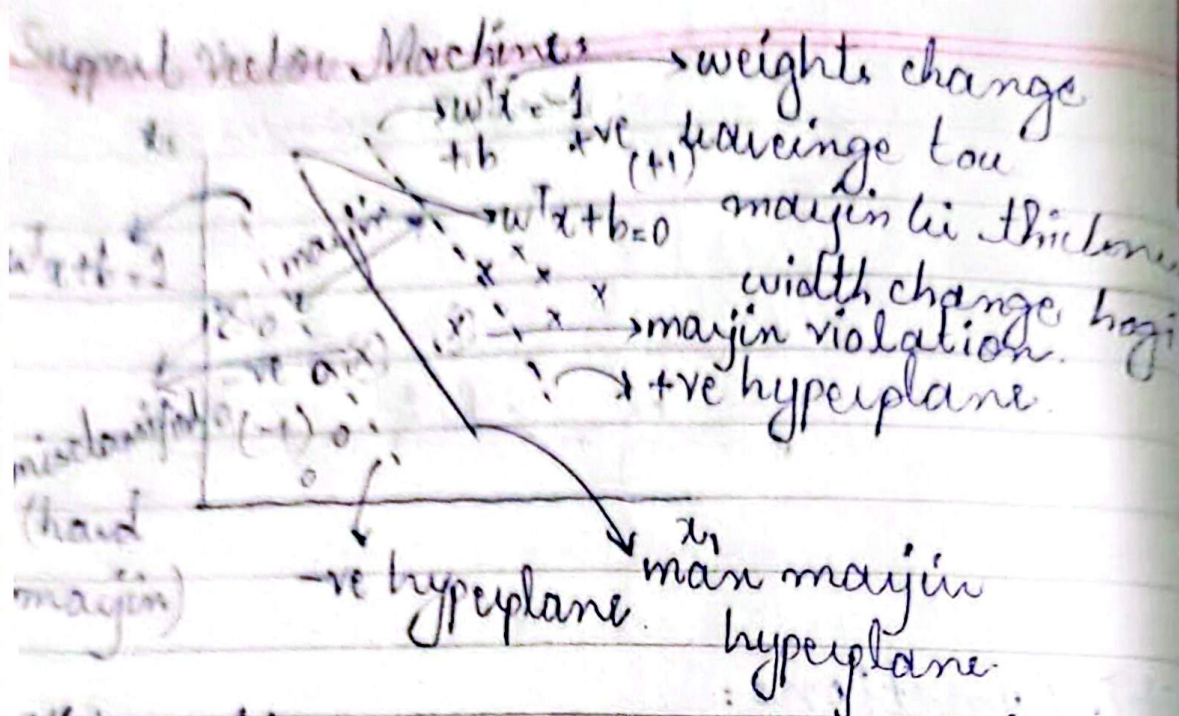
for max margin, we need to make w as
small as possible.

minimize $\frac{1}{2} \|w\|^2$ $\rightarrow$ weight vector

where, $\|w\|$ is L2 norm which is eq. to
 $\sqrt{w_1^2 + w_2^2 + \dots + w_m^2}$

- $\rightarrow$ for penalizing incorrect classification,
(Hinge loss).

$L(y^{(i)}, \hat{y}^{(i)}) = \max(0, 1 - y^{(i)}(w^T x^{(i)} + b))$



High margin: (heavily update)
misclassified $\rightarrow$ more update weights.
margin violation $\rightarrow$ less update weights.

Objective:
fun.

Batch 4.0 hinge loss

$$J(w, b) = \text{minimize } \frac{1}{2} \|W\|^2 + C \sum_{i=1}^n \left[1 - y^{(i)} (w^T x^{(i)} + b) \right]$$

in +ve region $\rightarrow$ max $\downarrow$ $[1 - 1(1.5)] = -0.5$
in margin $\uparrow$ $(0-1) \rightarrow [1 - 1(0.5)] = 0.5$
 $[1 - 1(-1.5)] = 2.5$

* Try for $y^{(i)} \rightarrow -1$

+ve
misclassified

support vectors $\rightarrow$ samples that lie on
+ve & -ve hyperplane of margin.

Hyperparameters: —

less penalizes violations
small $C \rightarrow$ wider margin, more margin
violations.

large $C \rightarrow$ narrow margin, lesser margin
violations.
penalizes violations heavily.

$C \rightarrow +\infty$ (hard margin classifier).

$$\lambda = \frac{1}{C}$$

$C \rightarrow$ for reg. (choti se chota
sakshya hai).

Hyperparameters

choice of kernel variables used in kernel func. (i.e. d & γ).

NON LINEAR SVM CLASSIFIER: -

Adding polynomial features.

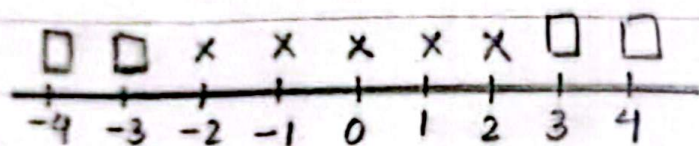
1 $\rightarrow$ $\square$
0 $\rightarrow$ x

x_1	-4	-3	-2	-1	0	1	2	3	4
x_2	16	9	4	1	0	1	4	9	16
y	1	1	0	0	0	0	0	1	1

$\rightarrow x_1^2, x_2$

1 feature: -

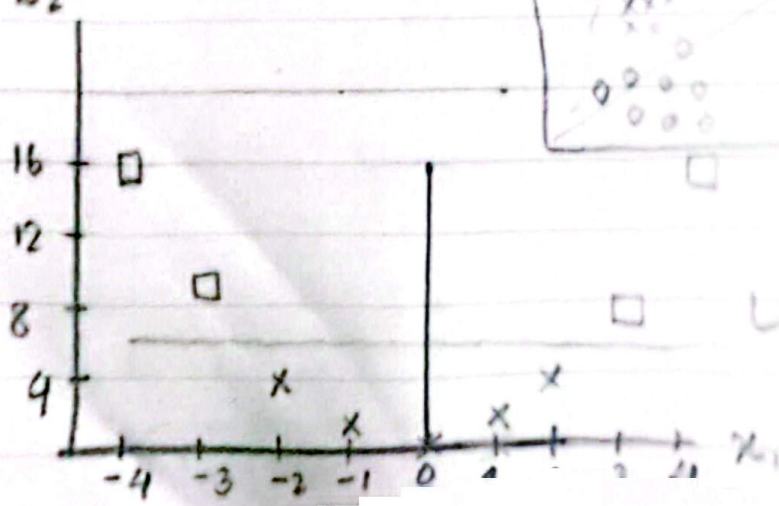
$\rightarrow$ Not linearly separable



Example: 2

greater polynomials becomes exponentially large

2 features: -



Linearly Separable

Kernel trick $\rightarrow$ Makes features linearly separable without adding polynomial features.
 through $\alpha \rightarrow$ tells which are support ^{vectors} which are ^{raising} not if $\alpha > 0 \rightarrow$ support vectors

SVM KERNELS $\rightarrow$ * Kernel func: measures similarity
 SVM identifies support vectors during training via learned coefficients α .

$\rightarrow$ During prediction the classifier evaluates a weighted sum of kernel based similarities between the test point and the support vectors to make the final decision.

$\rightarrow$ for non linear SVM. ^{new sample} (test point)

$$\hat{y} = \text{sign} \left(\sum_{i=1}^n \alpha^{(i)} y^{(i)} [k(x^{(i)}, x)] + b \right)$$
_{no values init α zero hua wo sum ka point nhi}

Types of Kernel $\rightarrow$ $\hat{y} = \text{sign}(w^T x + b)$ for linear SVM.

1) Linear kernel. ^{being use (coeff. of i^{th} sample)}
 $\rightarrow$ no kernel used (linear SVM).

2) Polynomial kernel $\rightarrow k(x, l) = (x^T l + c)^d$
_{degree of polynomial}

3) Radial basis function (RBF) kernel $\rightarrow$ Gaussian kernel

$$k(x, l) = \exp \left(-\frac{\|x - l\|_2^2}{2\sigma^2} \right) = \exp \left(-\gamma \|x - l\|_2^2 \right)$$

$\downarrow$ $\downarrow$ $\downarrow$
 $2\sigma^2$ $L2 \text{ norm}$ $\gamma = \frac{1}{2\sigma^2}$
 variance $\downarrow$ $\downarrow$ $\downarrow$
 S.D $\downarrow$ $\downarrow$ $\downarrow$
 variance $\downarrow$ $\downarrow$ $\downarrow$
 small γ - overcomes overfitting

4) Sigmoid kernel.

5) String kernel.

* checking similarities of test point with all support vectors

from handouts

SVM REGRESSION:-

→ We want the values/samples to be in margin

→ The difference b/w the $\hat{y}$ & y should not be greater than ϵ . (it would be depicting that sample is far away from margin).

$$(y^{(i)}, \hat{y}^{(i)}) =$$

→ combining many classifiers of different strengths
ENSEMBLE LEARNING →

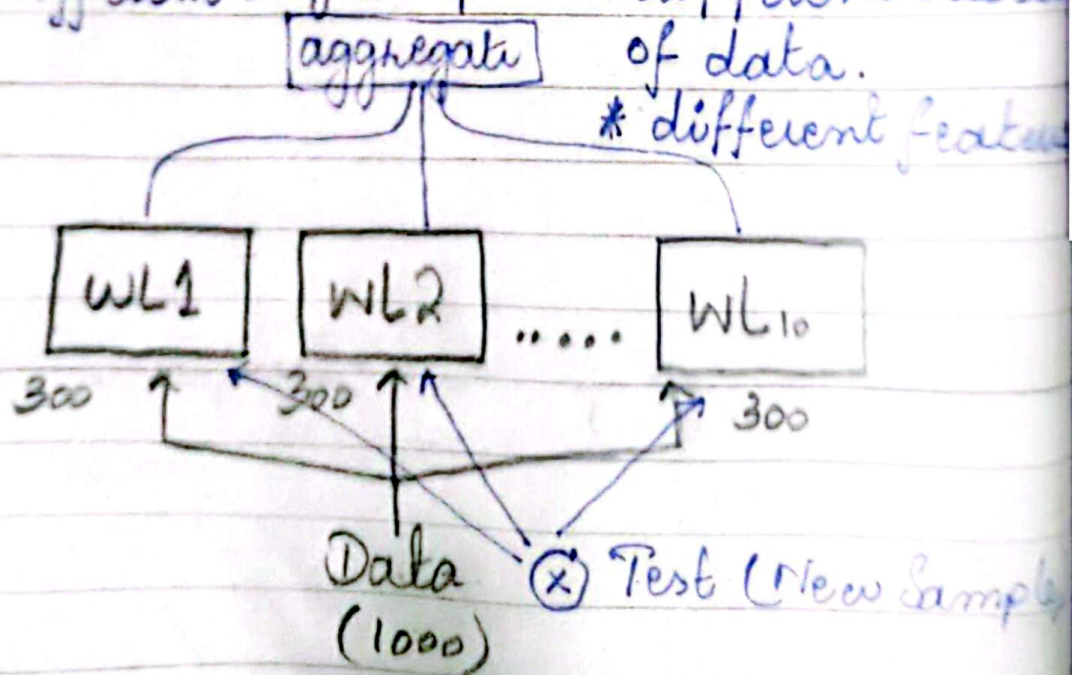
→ Ensemble learning combines predictions from multiple weak learners to produce more accurate and robust strong learners.

→ Output is aggregated:

Regression: take average i.e. mean, mode, median.

Classification: majority voting.

→ The strength of an ensemble lies in diversity:
(diversity of classifiers)
different algos. → o/p * different subset of data.
* different features



1. VOTING CLASSIFIERS:—

Diversity: Different algos.

→ Same data. (of every classifier).

Hard voting: Voting on 1's & 0's (hard 1 & 0 c/p's are voting).

Soft voting: Voting probabilities are averaged.

2. BAGGING & PASTING:—

Diversity: Different subsets of data are used to train weak learners.

→ Same algos. (of every classifier).

Bagging: bootstrap aggregating

* sampling with replacement.

* a sample can appear multiple times in a dataset.

Out-of-bag (OOB) error

Especially in bagging.
Bagging: Sampling without replacement.
* each sample appears only once in a data subset.

Ensembles based on Decision Trees: -
(most popular weak learners)

Limitation of DT:

* greedy.

* Overfitting issues when the DT is deep.
(we use pruning or early stopping).

* high variance: small changes in data drastically changes the DT structure.

Ensemble of DT

RANDOM FOREST: typically:
(DT + bagging)

→ Different features.

* Each DT can be trained on diff. machine
combine later. (parallel machine training)
hyperparameters.

all that of DT.

of DT.

of features.

bagging / pasting → if bootstrap = True
= False

Image segmentation $\rightarrow$ Similar pixels are placed together.

CLUSTERING — samples

$\rightarrow$ Separates data but can't label it because it is unsupervised learning.

K-means clustering:

$\rightarrow$ kitno clusters banana hai.
 $\rightarrow$ har cluster ke sath centroid hai.
associate hoga, naya sample centroid se distance find out karega aur jiska distance sab se kam hoga, that sample will belong to that cluster, hum randomly samples se centroid pick karenge (initially).

$\rightarrow$ Phir tamam samples ka mean calculate karke centroid relocate kar denge.

Stopping Condition:

$\rightarrow$ Jab centroid move nhi hua.

$\rightarrow$ Jab clusters ki location change nhi horegi.
value of U
 μ_k = centroid.

1) Cluster assignment step:

$$C(i) = \min_k \|x^{(i)} - \mu_k\| \quad \rightarrow \text{L2 norm (Euclidean distance)}$$

2) Move centroid:

Update each centroid by computing the mean of all points assigned to that cluster.

→ agar kisi se ...
hai m-dimensional hoga.

distance from centroid & its
Cost func. :: $\sum_{j=1}^K \sum_{i=1}^n \|x_i^{(j)} - \mu_j\|^2$

$$J = \sum_{j=1}^K \sum_{i=1}^n \|x_i^{(j)} - \mu_j\|^2$$

$4-2+2 \}$ PP
 $(2-1+1) \}$

μ_1 & μ_2 se min(,) ka dist. check krni
jo kam hai hoga, wo use cluster se belong
karega.

from handon

DECISION TREES:—

- For simple tabular data.
- It can do both classification & regression tasks.
- Decision trees are white box model, because their decisions are easy to interpret.
- No need for normalization / feature scaling.
- We can also work with categorical variable.
- Label can also be repeated (in leaf nodes, but one feature at a time).
- Leaf nodes ^{label} make labels homogeneous.

axis aligned → ya tou x-axis ke sath align hoga ya y-axis ke sath.

- In regression, each leaf node represent a numerical value rather than label.

Learning a decision tree:-

Algo
→ CART & ID3.

CART (Classification & Regression Trees).

- 1) Evaluate all possible combination of feature & threshold (which gives best impurity).
- 2) Divide into two subtrees.
- 3) Do this recursively.
- 4) Must be a stopping condition.
- 5) Assign label to classification or an average value (regression) to each leaf node.

Feature choosing → randomly or through PCA (pick priority wise).

Threshold choosing → Midpoint of feature.

- → if these threshold are too many values sort the feature take that midpoint for threshold where class is changing.

Impurity in Decision Trees & Pure Nodes

→ Uncertainty in o/p $\rightarrow$ Pure node

Highly impure $\rightarrow$ If It contains many different classes in node similar propagation

For checking purity; ~~Entropy~~ Gini impurity, entropy, MSE.

Stopping Conditions:

When decision tree becomes large it becomes large/complex & leads to over-fitting.

$\rightarrow$ Pre pruning.

$\rightarrow$ Post pruning.

* Left last hi 2 nodes pure when been

Week 15 lecture 1

Date:

Decision Trees

Impurity Reduction Metrics

MSE for regression.

Gini Index for classification and Entropy for classification.

If D is the dataset / subset of samples containing c classes.

$$gini(D) = 1 - \sum_{i=1}^c p_i^2 \quad \text{probability}$$

- If D is split into subsets D_1 and D_2 , weighted gini impurity of the split:

$$gini(D, S) = \frac{D_1}{D} gini(D_1) + \frac{D_2}{D} gini(D_2)$$

- condition:

$$gini(D, S) < gini(D)$$

Agar split karne ke baad ki impurity $<$ $gini(D)$ to hum split karne wala nahi karne wala.

Example:

Past trend (pt)	Open-interest (oi)	trading volume (tv)	return (ret)
[P → positive x_1 N → negative]	[L → low x_2 H → high]	[L → low x_3 H → High]	[U → up y D → Down]
1 P	L	H	U
2 N	H	L	D
3 P	L	H	U
4 P	H	H	U
5 N	L	H	D
6 P	L	L	P
7 N	H	H	D
8 N	L	H	D
9 P	L	L	D
10 P	H	H	U

Decision Tree

root Depth=0

D=10
gini=0.48
pt=P?

True P

False

D=6
gini=0.44
tv=L

D=4
gini=0
class=Don
parent node

leaf

(D)

gini=0
gini=0

(repeat and complete

or: this

gini (oi=L)

gini (oi=H) ✓

weighted

tv = 0, 0, 0

pt = 0.44, 0, 0.44 X

Depth 0

Gini Index of root node

$$P(\text{ret} = U) = 4/10$$

$$P(\text{ret} = D) = 6/10 \text{ Data}$$

$$gini = 1 - \left(\left(\frac{4}{10} \right)^2 + \left(\frac{6}{10} \right)^2 \right) = 0.48$$

For past trend (pt)

(D)

$$P(\text{pt} = P) = 6/10$$

$$P(\text{pt} = N) = 4/10$$

$$P(\text{ret} = U | \text{pt} = P)$$

Har data upi and down

count karway hongay.

$$P(\text{ret} = D | \text{pt} = P) = 2/6$$

$$P(\text{pt} = N) = 4/10$$

$$P(\text{ret} = U | \text{pt} = N) = 0/4$$

$$P(\text{ret} = D | \text{pt} = N) = 4/4$$

$$gini = 1 - \left(\left(\frac{4}{6} \right)^2 + \left(\frac{2}{6} \right)^2 \right)$$

$$= 0.44 \text{ (for pt=P)}$$

$$gini = 1 - ((0)^2 + (1)^2)$$

$$= 0 \text{ (for pt=N)}$$

weight gini index after the

$$\text{split} = \frac{6}{10} \times 0.44 + \frac{4}{10} \times 0$$

$$= 0.264 < 0.48$$

(Yeh weight average parent say kam honi chahiya)

For open interest (oi)

$$P(oi=L) = 6/10$$

$$P(oi=H) = 4/10$$

$$P(ret=U | oi=L) = 2/6$$

$$P(ret=U | oi=H) = 2/4$$

$$P(ret=D | oi=L) = 4/6$$

$$P(ret=D | oi=H) = 2/4$$

$$gini = 0.44$$

$$gini = 0.5$$

$$\text{Weighted gini index after split} = 0.464 < 0.48$$

For trading Volume (tv)

$$P(tv=L) = 3/10$$

$$P(tv=H) = 7/10$$

$$gini = 0.49$$

$$gini = 0$$

$$\text{Weighted} = 0.343 < 0.48$$

Agar tree ko grow karney mein kay saray nodes pure hojayein end pay to it will highly likely cause overfitting.

overfitting ka sabay masla tree mein hota so you put some stopping conditions:

① Depth set kardo as stopping condition.

Agar aisey node pay ruk rahay jo 0 nahi to majority voting karkey class assign kardain.

② You can also fix the number of samples.

Hyperparameters:

① which metric to use

② stopping conditions.